

Lab2

STAT 4610/5610 Spring 2026

2026-02-20

Lab2: Comparing Nonlinear Regressions

Due: 11:59PM Friday, February 27 on Canvas as a knitted pdf file of your team's Quarto document

1. You will individually fit **six** nonlinear regression models on a single X variable from your team's chosen dataset. You will use CV (including the built-in LOOCV functions in R) on at least three of these models to determine the best tuning parameters (e.g., λ , K number of knots or steps, or degrees of freedom). You will compare the six models, choose the best one, make a prediction at an important value of X, interpret the results, and comment on the ethics of the situation.
2. Teams will use the individual contributions to build a Generalized Additive Model (GAM), fit it, make predictions, make interpretations, show visualizations, etc.
3. Each individual will comment on how they contributed to their team's GAM.

Instructions for Lab2

In HW2, you applied several types of shrinkage and selection methods for linear models (shrinkage and selection methods) on a dataset of your choosing. This lab will be similar, but you will apply the nonlinear regression methods of Chapter 7 in *ISLR2* and use cross validation to choose the tuning parameters and select the best method. Individuals will model individual variables from the same dataset (one variable for each team member) and then combine their work into one team GAM that uses multiple X variables to predict Y.

What teams need to do first

1. Choose a dataset that has enough interesting X variables that you can combine to model/predict Y. You can use one of the datasets that a teammate used for HW2 or pick a new one. You cannot use the Wage data from *ISLR2*. Auto Data Set: Predicting Y-MPG

2. Decide who will model which X variable individually and when they need to be done to contribute to the team GAM. Me-Acceleration

What individuals need to do

1. Given your team's dataset and your X variable, fit a polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and a local regression (six models).
2. Use CV to determine the optimal tuning parameters for at least three of your models. Sometimes just choosing the tuning parameter (e.g., picking the number of cuts in X for a step function regression) based on your knowledge of Q1 and Q3 makes more sense than using CV. *Use LOOCV
3. Compare the CV MSE or other appropriate model summary (e.g., misclassification rate) for your models. Choose the best model method.
4. At an important or significant value of X (explain why that value of x0 is important or significant), use your model to predict Y at that value.
5. Interpret your prediction and generally what the model is telling you about Y.
6. Comment on any ethical implications of this work.
7. Optional: Create visualizations of your models or just of the best model.

What teams need to do

1. Create a GAM based on the work of your individual team members.
2. Visualize the GAM (R has some good built-in plotting functions, such as `plot.gam()`).
3. Choose a value of X that is important or interesting (explain why) and use that point to make a prediction for Y. Interpret this prediction.
4. How does the GAM compare with a linear method (e.g., lasso, ridge, PCR, etc.)?

Individual contributions to team GAM

Each individual must comment on their contributions to the team GAM. For example, write something like: 'I told person X that my results showed that the natural spline with 4 knots was the best model for variable x1. Then I created a visualization to help interpret the GAM.' Or, 'I used the GAM to make a prediction for the point x0.' Or, 'I didn't actually do anything for the team section.' Only individuals who contribute to the team section will get points for the team section.

Some intended outcomes from this assignment:

- You will individually get practice fitting a nonlinear regression models: polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and local regression

- You will practice using CV to select the optimal tuning parameters of your models
- You will use CV to compare the performance of these models
- You will make predictions based on your models
- You will practice fitting and interpreting a GAM
- You will gain experience working on a data science “project” as a team

Emery Individual Section:

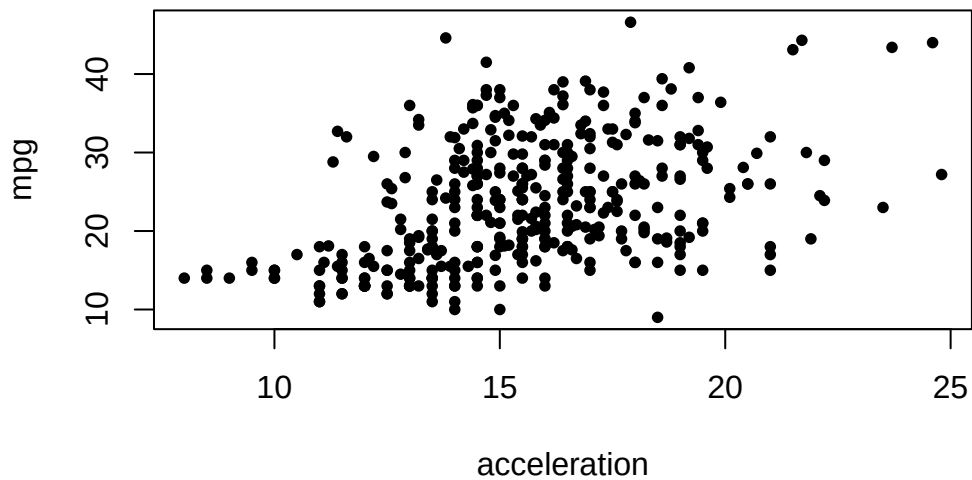
```
library(ISLR2)
library(boot)
library(splines)
library(ggplot2)
set.seed(0)

# Data
dat <- Auto
dat <- na.omit(dat)

Y <- "mpg"
X <- "acceleration"
```

```
plot(dat$acceleration, dat$mpg,
     xlab = "acceleration",
     ylab = "mpg",
     main = "Auto: mpg vs acceleration",
     pch = 16, cex = 0.8)
```

Auto: mpg vs acceleration



Polynomial Regression

```
deg_grid <- 1:10
poly_mse <- rep(NA, length(deg_grid))
poly_fit_list <- vector("list", length(deg_grid))

for(i in seq_along(deg_grid)){
  d <- deg_grid[i]
  fit <- glm(mpg ~ poly(acceleration, d, raw = TRUE), data = dat)
  poly_fit_list[[i]] <- fit
  poly_mse[i] <- cv.glm(dat, fit)$delta[1]
}

best_deg <- deg_grid[which.min(poly_mse)]
best_poly_fit <- poly_fit_list[[which.min(poly_mse)]]

best_deg
```

```
[1] 4
```

```
min(poly_mse)
```

```
[1] 49.40298
```

Step Function

```
k.grid <- 2:10
step.mse <- rep(NA, length(k.grid))
step.fits <- vector("list", length(k.grid))

for(i in seq_along(k.grid)){
  k <- k.grid[i]

  # 1) Make fixed breaks using the FULL dataset (so they don't change in LOOCV)
  breaks.k <- seq(min(dat$acceleration), max(dat$acceleration), length.out = k + 1)

  # 2) Create the step-function factor ONCE using those fixed breaks
  dat$acc.step <- cut(dat$acceleration, breaks = breaks.k, include.lowest = TRUE)

  # 3) Fit model using the precomputed factor
  fit <- glm(mpg ~ acc.step, data = dat)
  step.fits[[i]] <- fit

  # 4) LOOCV
  step.mse[i] <- cv.glm(dat, fit)$delta[1]
}

best.k <- k.grid[which.min(step.mse)]
best.step <- step.fits[[which.min(step.mse)]]

best.k
```

[1] 3

```
min(step.mse)
```

[1] 49.08606

Cubic Spline

```
df.grid <- 3:12
cubic.mse <- rep(NA, length(df.grid))
cubic.fits <- vector("list", length(df.grid))

for(i in seq_along(df.grid)){
```

```

df0 <- df.grid[i]
fit <- glm(mpg ~ bs(acceleration, df = df0), data = dat)
cubic.fits[[i]] <- fit
cubic.mse[i] <- cv.glm(dat, fit)$delta[1] # LOOCV
}

```

Warning in bs(acceleration, degree = 3L, knots = numeric(0), Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = numeric(0), Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = numeric(0), Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = numeric(0), Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = 15.5, Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = 15.5, Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = 15.5, Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = 15.5, Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(14.5, 16.5), Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(14.5, 16.5), Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(14.5, 16.5), Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(14.5, 16.5), Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.8, 15.5, 17.05),
Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.8, 15.5, 17.05),
Boundary.knots = c(8.5, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.75, 15.5, 17),
Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.75, 15.5, 17),
Boundary.knots = c(8, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.4, 14.8, 16, 17.7), :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(13.4, 14.8, 16, 17.7), :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13.5, 14.8, 16, 17.7), :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(13.5, 14.8, 16, 17.7), :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13, 14.5, 15.5, 16.5, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(13, 14.5, 15.5, 16.5, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(13, 14.5, 15.5, 16.5, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(13, 14.5, 15.5, 16.5, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.8714285714286, 14, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.8714285714286, 14, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.8, 14, 15, 15.8, 16.9, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.8, 14, 15, 15.8, 16.9, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.8, 14.5, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.8, 14.5, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.75, 14.5, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.75, 14.5, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.3, 13.5, 14.5, 15, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.3, 13.5, 14.5, 15, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.5, 14.5, 15, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12.5, 13.5, 14.5, 15, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12, 13.5, 14.1, 14.8, : some
'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12, 13.5, 14.1, 14.8, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(acceleration, degree = 3L, knots = c(12, 13.4, 14, 14.8, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(acceleration, degree = 3L, knots = c(12, 13.4, 14, 14.8, 15.5, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

```
best.df.cubic <- df.grid[which.min(cubic.mse)]  
best.cubic <- cubic.fits[[which.min(cubic.mse)]]  
  
best.df.cubic
```



```
[1] 4
```

```
min(cubic.mse)
```

```
[1] 49.10355
```

Natural Spline

```
knots.ns <- quantile(dat$acceleration, c(0.25, 0.5, 0.75))  
knots.ns
```

```
      25%    50%    75%  
13.775 15.500 17.025
```

```
ns.fit <- glm(mpg ~ ns(acceleration, knots = knots.ns), data = dat)  
ns.mse <- cv.glm(dat, ns.fit)$delta[1]  
  
ns.mse
```

```
[1] 48.96379
```

Smoothing Spline

```
ss.fit <- smooth.spline(dat$acceleration, dat$mpg, cv = TRUE)
```

```
Warning in smooth.spline(dat$acceleration, dat$mpg, cv = TRUE):  
cross-validation with non-unique 'x' values seems doubtful
```

```
ss.pred <- predict(ss.fit, x = dat$acceleration)$y  
ss.mse <- mean((dat$mpg - ss.pred)^2)  
  
ss.fit$df
```

```
[1] 4.636823
```

```
ss.mse
```

```
[1] 47.83098
```

Local Regression

```

span.grid <- c(0.2, 0.3, 0.4, 0.5, 0.6, 0.75, 1.0)
loess.mse <- rep(NA, length(span.grid))
n <- nrow(dat)

for(s in seq_along(span.grid)){
  sp <- span.grid[s]
  errs <- rep(NA, n)

  for(i in 1:n){
    fit <- loess(mpg ~ acceleration, data = dat[-i, ], span = sp)
    pred <- predict(fit, newdata = dat[i, ])
    errs[i] <- (dat$mpg[i] - pred)^2
  }

  loess.mse[s] <- mean(errs, na.rm = TRUE)
}

best.span <- span.grid[which.min(loess.mse)]
best.loess <- loess(mpg ~ acceleration, data = dat, span = best.span)

best.span

```

```
[1] 0.5
```

```
min(loess.mse)
```

```
[1] 48.80702
```

Results

```

results <- data.frame(
  model = c(
    paste0("Polynomial (degree=", best_deg, ")"),
    paste0("Step function (intervals=", best.k, ")"),
    paste0("Cubic spline (df=", best.df.cubic, ")"),
    paste0("Natural spline (knots at 25/50/75% quantiles)"),
    paste0("Smoothing spline (df~", round(ss.fit$df, 2), ")"),
    paste0("Local Regression (span=", best.span, ")")
  ),
  loocv.mse = c(

```

```

    min(poly_mse),
    min(step.mse),
    min(cubic.mse),
    ns.mse,
    ss.mse,
    min(loess.mse)
  )
)

```

```
results[order(results$loocv.mse), ]
```

	model	loocv.mse
5	Smoothing spline (df~4.64)	47.83098
6	Local Regression (span=0.5)	48.80702
4	Natural spline (knots at 25/50/75% quantiles)	48.96379
2	Step function (intervals=3)	49.08606
3	Cubic spline (df=4)	49.10355
1	Polynomial (degree=4)	49.40298

- The best model for predicting based on acceleration is a smoothing spline with approximately 5 degrees of freedom.

Predict an important value of X

```

x0 <- 15.5 # median acceleration (roughly central value)

predict_ss <- predict(ss.fit, x = x0)$y

predict_ss

```

```
[1] 24.58601
```

I selected the median acceleration (15.5 seconds) as a meaningful value because it represents the center of the dataset and reflects a typical vehicle's performance. Using the best model chosen by the lowest LOOCV MSE across the six models—the smoothing spline—the predicted fuel efficiency at this acceleration value is approximately 24.5 mpg. This prediction is very close to the overall average mpg of the dataset (23.45 mpg), suggesting that the model produces reasonable and realistic estimates for a typical vehicle.

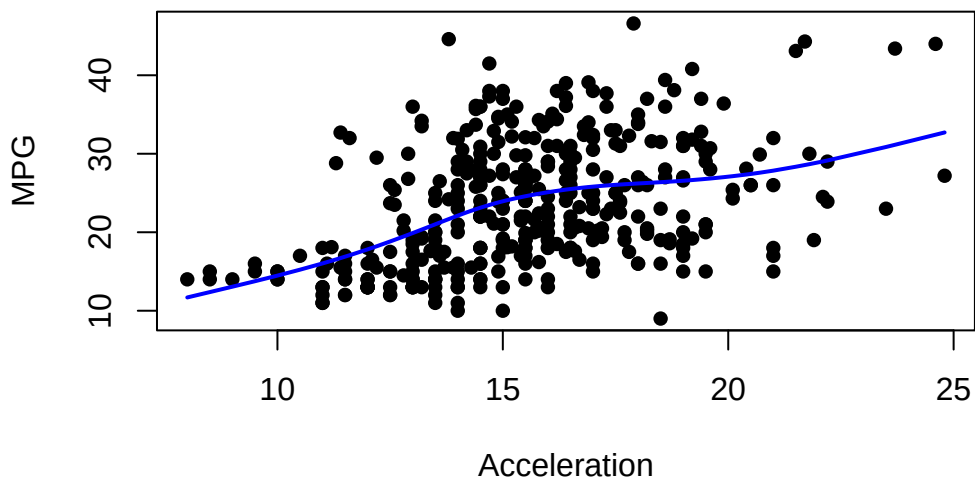
Ethical Implications

Modeling fuel efficiency has ethical implications because predictions may influence consumer decisions and environmental policy. If models are overfit or misinterpreted, they could mislead

stakeholders, and lead to poor policy or business decisions. Additionally, vehicle performance data may not account for real-world driving behavior or emissions standards.

Visualization for Best Model (Smoothing Spline)

```
plot(dat$acceleration, dat$mpg,  
     pch = 16,  
     xlab = "Acceleration",  
     ylab = "MPG")  
  
lines(ss.fit, col = "blue", lwd = 2)
```



```
rm(list = ls())
```

Jake Individual Section

Using the ILSR2 Auto dataset, I will regress miles per gallon (mpg) onto engine displacement (measured in cubic inches) using six non-linear models: polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and local regression.

I will use LOOCV to choose hyperparameters for four methods: step function regression, cubic spline, natural spline, and smoothing spline. For each of these, I will choose the number of evaluate which number of breaks/knots yielding the lowest LOOCV MSE.

Fitting and evaluating all 6 models

```
set.seed(0)
head(Auto)
```

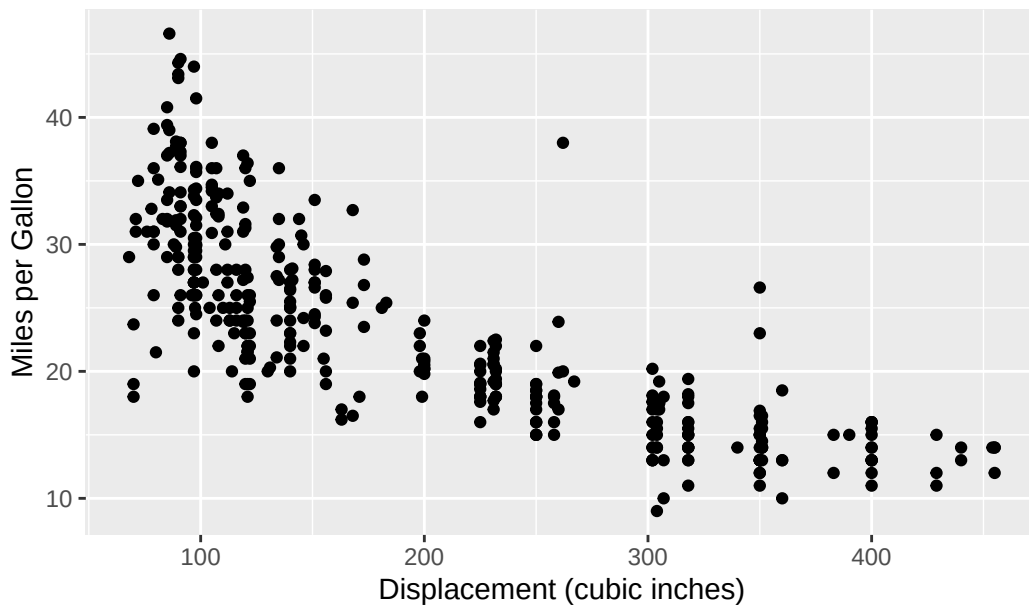
	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
1	18	8	307	130	3504	12.0	70	1
2	15	8	350	165	3693	11.5	70	1
3	18	8	318	150	3436	11.0	70	1
4	16	8	304	150	3433	12.0	70	1
5	17	8	302	140	3449	10.5	70	1
6	15	8	429	198	4341	10.0	70	1

	name
1	chevrolet chevelle malibu
2	buick skylark 320
3	plymouth satellite
4	amc rebel sst
5	ford torino
6	ford galaxie 500

```
n <- nrow(Auto)

# plot mpg vs displacement ggplot
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  labs(title = "MPG vs Displacement", x = "Displacement (cubic inches)", y = "Miles per Gallon")
```

MPG vs Displacement



Clearly, the relationship is non-linear. To me, it looks like a negative logarithmic relationship. And, I think that 1-3 knots will be sufficient for a good spline fit.

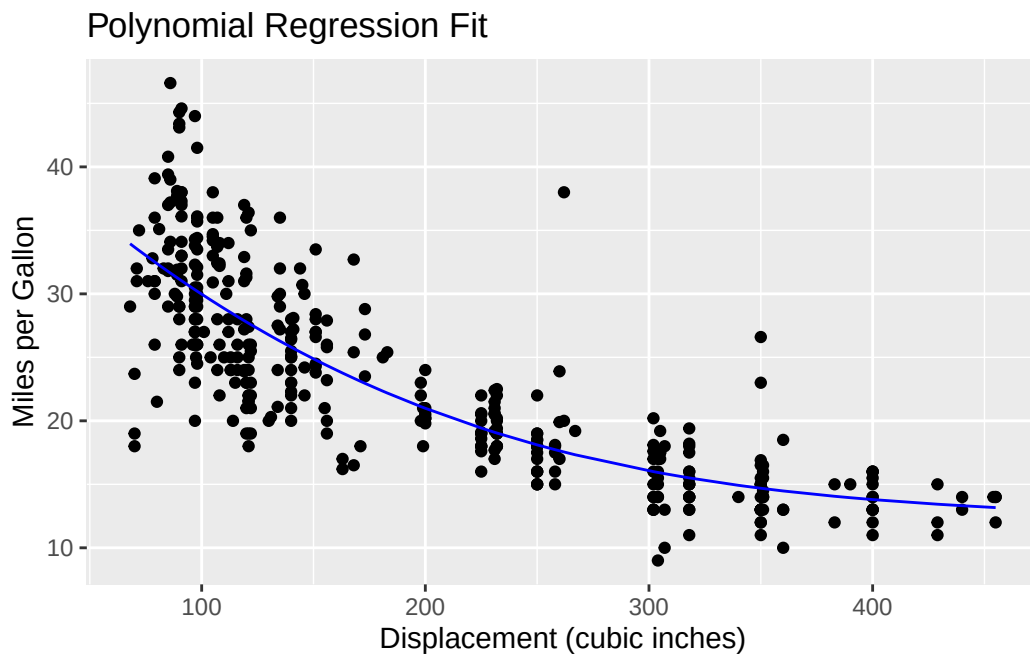
Also, I think a polynomial fit should have at least degree 3 because neither a linear nor quadratic fit would capture this relationship well. For fair comparison, I will use degree 3 for both polynomial and natural spline fits.

Overall, based on the plot, I expect all methods to perform well except step function regression because the rate of change of mpg, with respect to displacement, is very rapid for the first 50% of displacement values.

Polynomial regression

```
# polynomial regression
poly_all <- glm(mpg ~ poly(displacement, degree = 3), data = Auto)
# loocv for polynomial regression
cv_poly <- cv.glm(Auto, poly_all, K=n)
# get mse
mse_poly <- cv_poly$delta[1]
# plot model fit on all data
pred_poly <- predict(poly_all, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
```

```
geom_line(aes(y = pred_poly), color = "blue") +
labs(title = "Polynomial Regression Fit", x = "Displacement (cubic inches)", y = "Miles per
```



```
print(paste("Polynomial Regression MSE:", round(mse_poly, 2)))
```

```
[1] "Polynomial Regression MSE: 19.19"
```

Polynomial regression with degree 3 had a great visual fit and the MSE was 19.19 under LOOCV.

Step function regression

Given the visual relationship between displacement and mpg, I do not expect step function regression to perform well unless the number of breaks is very large. Therefore, I will try a small, medium, and large number of breaks where I expect the best fit to be with a large number of breaks.

```
# breaks is number of different pieces
breaks <- c(2, 5, 10)
mse_step <- c()
# breakpoints need to be extended to handle errors with cut()
```

```

delta <- 0.1
for (b in breaks) {
  breakpoints <- seq(min(Auto$displacement) - delta, max(Auto$displacement) + delta, length.out = b)
  step_model <- glm(mpg ~ cut(displacement, breaks = breakpoints), data = Auto)
  cv_step <- cv.glm(Auto, step_model, K=n)
  mse_step <- c(mse_step, cv_step$delta[1])
  print(paste("Step function regression with", b, "breaks MSE:", round(cv_step$delta[1], 2)))
}

```

```

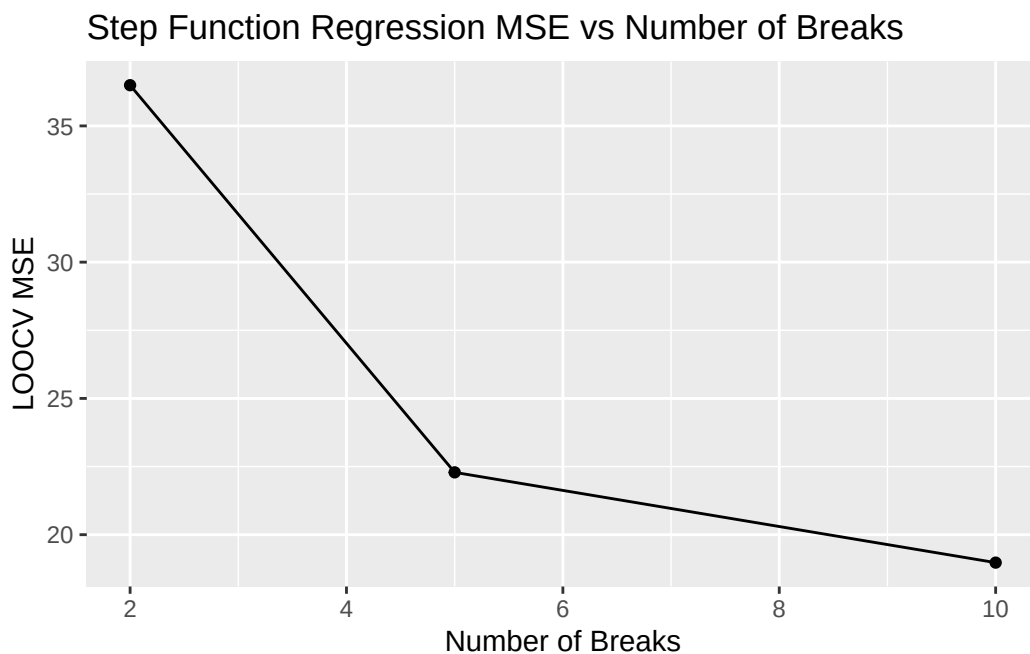
[1] "Step function regression with 2 breaks MSE: 36.49"
[1] "Step function regression with 5 breaks MSE: 22.29"
[1] "Step function regression with 10 breaks MSE: 18.98"

```

```

# plot breaks vs mse
ggplot(data.frame(breaks, mse_step), aes(x = breaks, y = mse_step)) +
  geom_point() +
  geom_line() +
  labs(title = "Step Function Regression MSE vs Number of Breaks", x = "Number of Breaks", y = "LOOCV MSE")

```

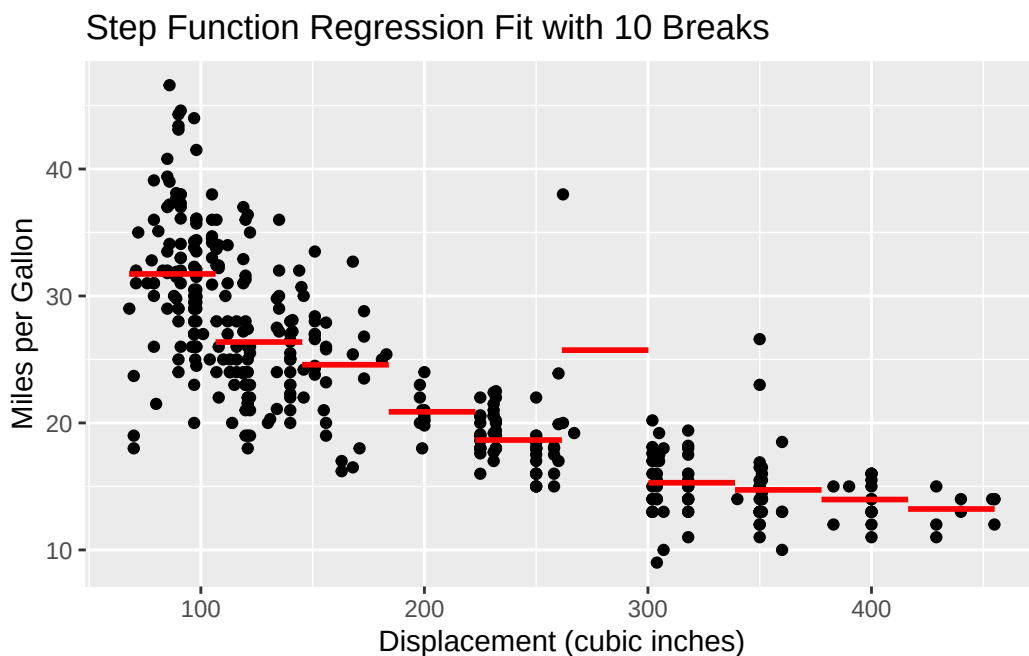


As expected, MSE was minimized for the largest number of breaks (10) with MSE of 18.98.


```
# plot model fit for 10 breaks
breakpoints <- seq(min(Auto$displacement) - delta, max(Auto$displacement) + delta, length.out = 11)
step_model <- glm(mpg ~ cut(displacement, breaks = breakpoints), data = Auto)

# get the piecewise constant values for each interval
step_data <- data.frame(
  x_start = breakpoints[-length(breakpoints)],
  x_end = breakpoints[-1],
  y = coef(step_model)[1] + c(0, coef(step_model)[-1])
)

ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_segment(data = step_data, aes(x = x_start, xend = x_end, y = y, yend = y),
    color = "red", linewidth = 1) +
  labs(title = "Step Function Regression Fit with 10 Breaks", x = "Displacement (cubic inches)
```



Clearly, the visual fit for step function regression is subpar compared to polynomial regression despite having better LOOCV MSE with 10 piecewise constants. LOOCV can underestimate error due to variance. For example, the outlier mpg point at $\sim (260, 37)$ caused an undesirable fit for the piecewise constant in its interval.

Cubic spline

Given the visual relationship between displacement and mpg, I think few to no knots are needed for a good fit. Since 0 knots corresponds to a degree 3 polynomial fit, I will just try adding 1-3 knots and see if it improves the fit. I will place the knots at the quantiles of the displacement variable.

```
knots <- quantile(Auto$displacement, probs = c(0.25, 0.5, 0.75))
mse_cubic <- c()
for (k in 1:3) {
  cubic_model <- glm(mpg ~ bs(displacement, knots = knots[1:k], degree = 3), data = Auto)
  cv_cubic <- cv.glm(Auto, cubic_model, K=n)
  mse_cubic <- c(mse_cubic, cv_cubic$delta[1])
  print(paste("Cubic spline with", k, "knots MSE:", round(cv_cubic$delta[1], 2)))
}
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105), Boundary.knots
= c(70, : some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105), Boundary.knots
= c(70, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
[1] "Cubic spline with 1 knots MSE: 18.61"
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151:
some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151:
some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
[1] "Cubic spline with 2 knots MSE: 17.75"
```

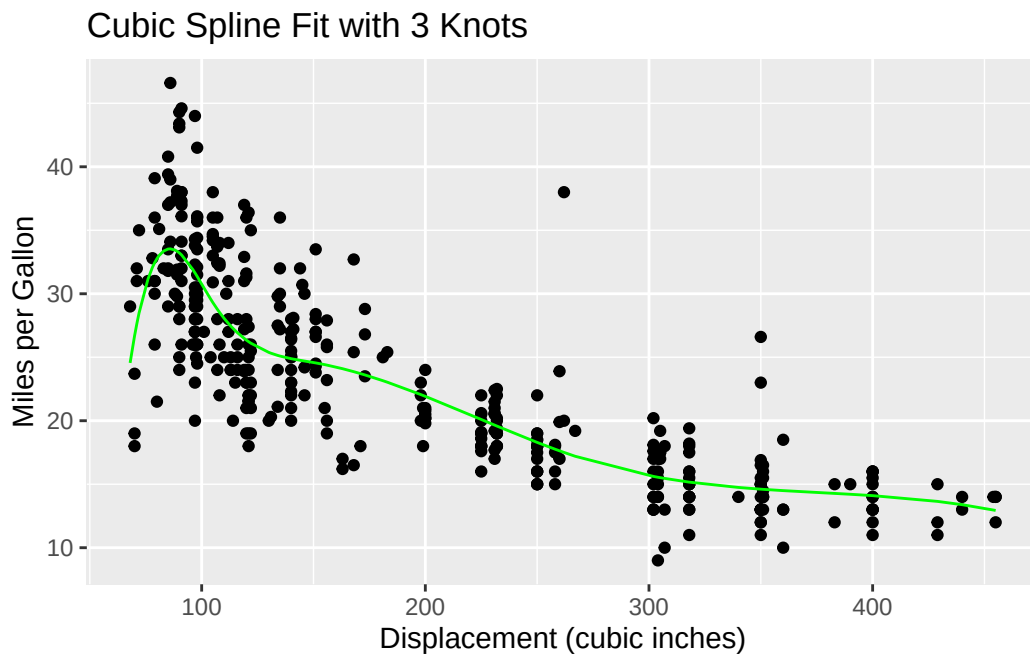
```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
[1] "Cubic spline with 3 knots MSE: 17.7"
```

Similar to step function regression, the most flexible cubic spline fit with 3 knots had the lowest MSE (17.7). The visual fit will reveal if overfitting is occurring.

```
# plot model fit for 3 knots
cubic_model <- glm(mpg ~ bs(displacement, knots = knots[1:3], degree = 3), data = Auto)
pred_cubic <- predict(cubic_model, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_cubic), color = "green") +
  labs(title = "Cubic Spline Fit with 3 Knots", x = "Displacement (cubic inches)", y = "Miles
```



From the visual fit, the main difference between the cubic spline and polynomial fit is that the cubic spline fits a different leading tail that minimizes some more error for the smallest displacement values.

Natural spline

For natural splines, I will also try 1-3 knots at the same quantiles as the cubic spline. MSE will probably be minimized with a larger number of knots. Again, I will check the visual fit to see if overfitting is occurring.

```
# natural spline uses degrees of freedom which is number of knots + 1
dfs <- c(2,3,4)

mse_natural <- c()
```

```

for (d in dfs) {
  natural_model <- glm(mpg ~ ns(displacement, df= d), data = Auto)
  cv_natural <- cv.glm(Auto, natural_model, K=n)
  mse_natural <- c(mse_natural, cv_natural$delta[1])
  print(paste("Natural spline with", d, "degrees of freedom MSE:", round(cv_natural$delta[1])
}

```

```

[1] "Natural spline with 2 degrees of freedom MSE: 19.1"
[1] "Natural spline with 3 degrees of freedom MSE: 19.2"
[1] "Natural spline with 4 degrees of freedom MSE: 19.36"

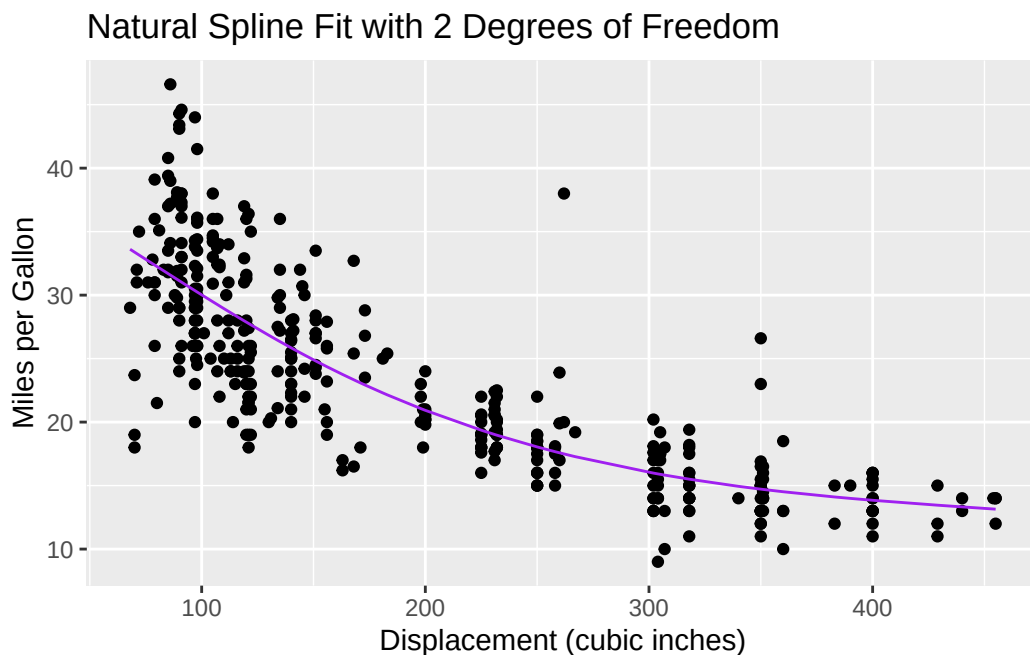
```

With degrees of freedom equal to 2 (1 knot) achieved the lowest MSE (19.1). I would guess the reason natural spline cannot overfit with more knots is because the linear tail constraint prevents the overfitting for small displacement values that we saw with the cubic spline fit.

```

# plot model fit for 2 degrees of freedom (1 knot)
natural_model <- glm(mpg ~ ns(displacement, df= 2), data = Auto)
pred_natural <- predict(natural_model, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_natural), color = "purple") +
  labs(title = "Natural Spline Fit with 2 Degrees of Freedom", x = "Displacement (cubic inches)

```



Smoothing spline

For hyperparameter tuning smoothing splines, the degrees of freedom do not correspond directly to the number of knots; instead, higher degrees of freedom just implies more smoothing flexibility. Therefore, I will try a small, medium, and large number of degrees of freedom to understand how the fit changes.

```
dfs_smooth <- c(2,5,10)

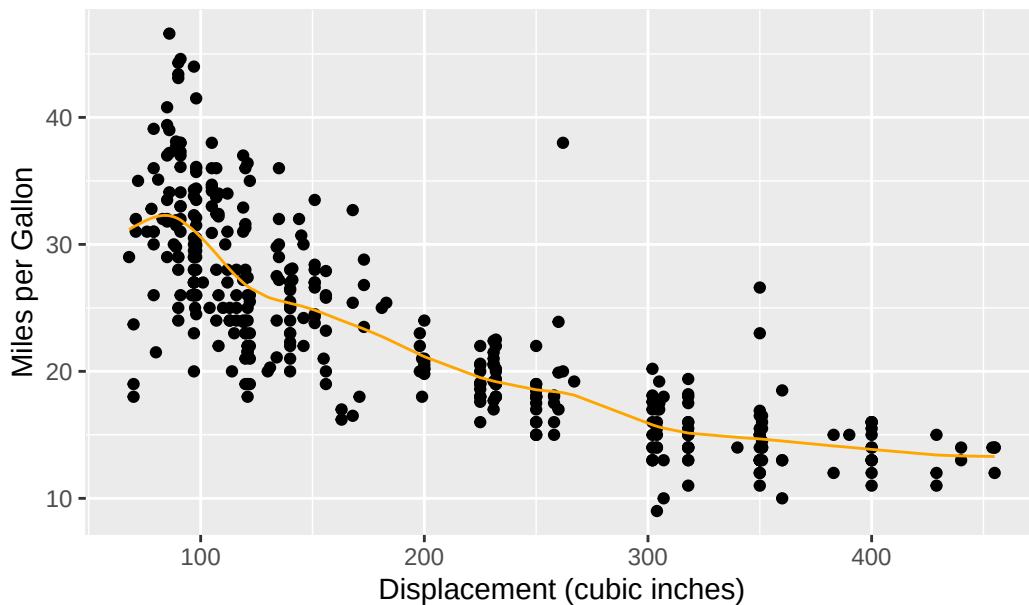
mse_smooth <- c()
for (d in dfs_smooth) {
  smooth_model <- smooth.spline(Auto$displacement, Auto$mpg, df = d)
  pred_smooth <- predict(smooth_model, x = Auto$displacement)$y
  mse_smooth <- c(mse_smooth, mean((Auto$mpg - pred_smooth)^2))
  print(paste("Smoothing spline with", d, "degrees of freedom MSE:", round(mean((Auto$mpg - p
})
```

```
[1] "Smoothing spline with 2 degrees of freedom MSE: 21.35"
[1] "Smoothing spline with 5 degrees of freedom MSE: 18.71"
[1] "Smoothing spline with 10 degrees of freedom MSE: 17.6"
```

The minimum MSE was 17.6 for the most smooth spline (10 df).

```
# plot model fit for 10 degrees of freedom
smooth_model <- smooth.spline(Auto$displacement, Auto$mpg, df = 10)
pred_smooth <- predict(smooth_model, x = Auto$displacement)$y
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_smooth), color = "orange") +
  labs(title = "Smoothing Spline Fit with 10 Degrees of Freedom", x = "Displacement (cubic in
```

Smoothing Spline Fit with 10 Degrees of Freedom



The visual fit is an intermediate between the cubic spline and natural spline fits. The fit is more flexible than the natural spline fit, but the overfitting for small displacement values is less severe than the cubic.

Local regression

I expect local regression to perform similarly to cubic polynomial and natural spline regression since the visual relationship can probably be approximated by a sequence of linear fits.

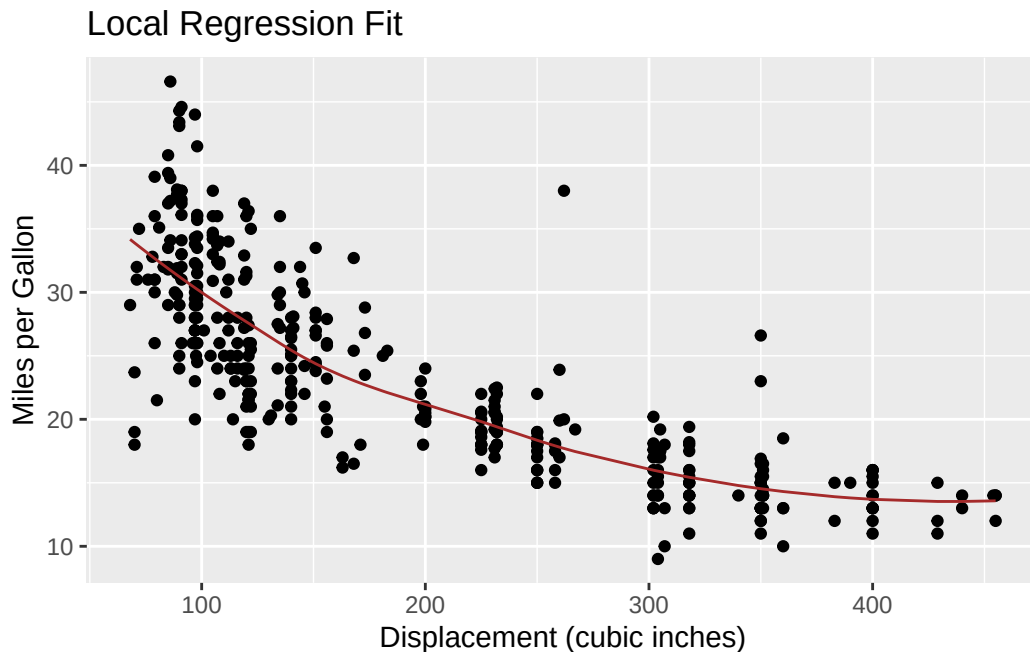
I am not tuning the span hyperparameter. I will use the default.

```
local_model <- loess(mpg ~ displacement, data = Auto)
pred_local <- predict(local_model, newdata = Auto)
mse_local <- mean((Auto$mpg - pred_local)^2)
print(paste("Local regression MSE:", round(mse_local, 2)))
```

```
[1] "Local regression MSE: 18.86"
```

The MSE for local regression was 18.86.

```
# plot local regression fit
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_local), color = "brown") +
  labs(title = "Local Regression Fit", x = "Displacement (cubic inches)", y = "Miles per Gal.
```



As expected, the local regression fit is similar to the cubic polynomial and natural spline fits.

Summary of 6 model evaluation

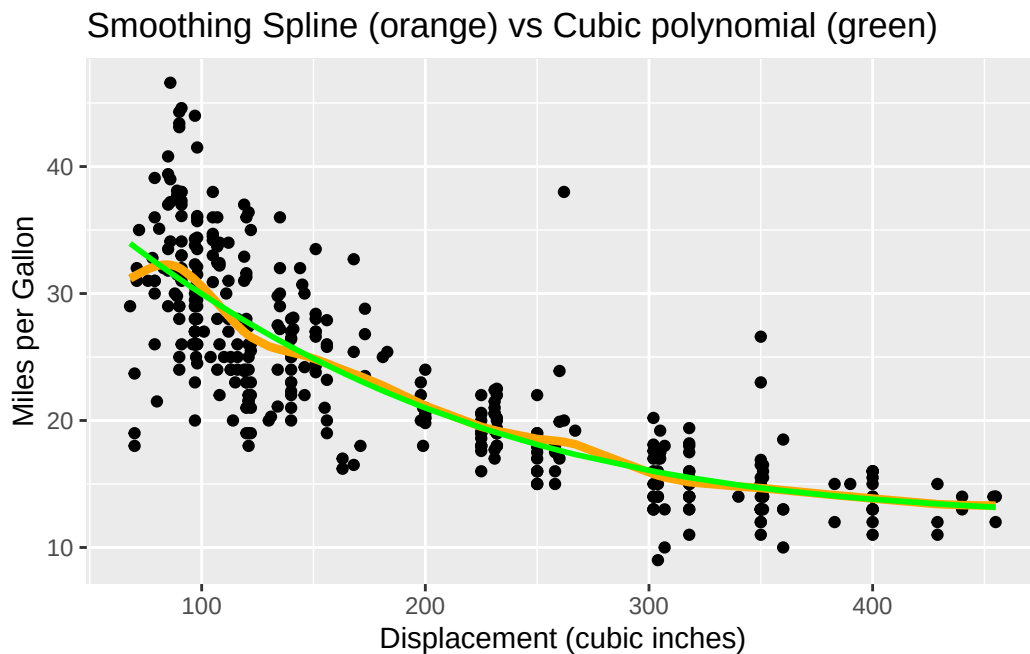
Across all 6 models and after hyperparameter tuning for 4 of the models, the lowest MSE was 17.6 for the smoothing spline with 10 degrees of freedom.

The instructions state to choose the best model from LOOCV MSE; however, I think the visualization of each model suggests that a simple cubic polynomial is probably the best option.

For completeness, I will plot both the smoothing spline and the cubic polynomial.

```
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_smooth), color = "orange", size = 1.5) +
  geom_line(aes(y = pred_poly), color = "green", size = 1) +
  labs(title = "Smoothing Spline (orange) vs Cubic polynomial (green)", x = "Displacement (c
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



The smoothing spline is more flexible and has lower MSE, but the cubic polynomial is simpler and has a visually good fit.

Prediction at significant X

A significant value of displacement (X) is 260 cubic inches because there exists a high leverage point near this value in the training data set. This high leverage point had significant impact on the step function regression fit, and it is interesting to consider the tradeoff between choosing a model that fits this point well or not.

```
# predict at x0 = 260 for both cubic polynomial and smoothing spline
x0 <- 260
pred_poly_x0 <- predict(poly_all, newdata = data.frame(displacement = x0))
pred_smooth_x0 <- predict(smooth_model, x = x0)$y
print(paste("Prediction at x0 = 260 for cubic polynomial:", round(pred_poly_x0, 2)))
```

```
[1] "Prediction at x0 = 260 for cubic polynomial: 17.64"
```



```
print(paste("Prediction at x0 = 260 for smoothing spline:", round(pred_smooth_x0, 2)))
```

```
[1] "Prediction at x0 = 260 for smoothing spline: 18.37"
```

Interpretation

The predicted mpg for 260 cubic inches is about 18 mpg for both cubic polynomial and smoothing spline.

In conclusion, I think it is better to fit the majority of points nearby 260 cubic inches well rather than overfitting to the high leverage point at 260.

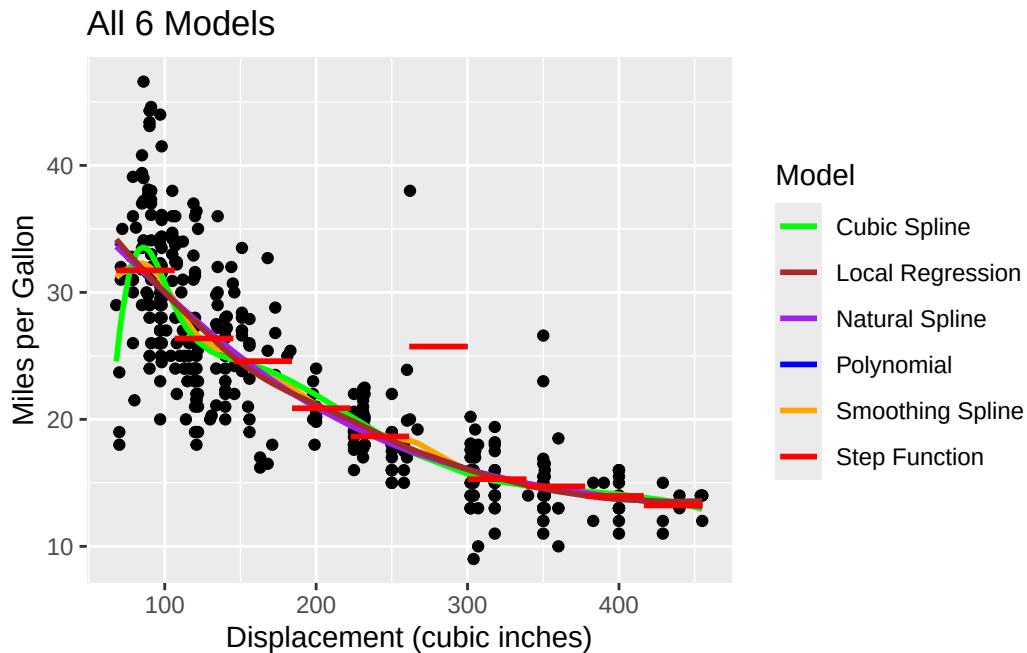
In general, the model shows that as engine displacement increases, miles per gallon decreases in a non-linear fashion.

Ethical implications

A potential ethical implication is that clearly displacement is not the only factor affecting miles per gallon. For an approximately fixed displacement value, there exists a range of mpgs in the training data. Therefore, this model is overly simplistic and this study is purely pedagogical. In practice, including more predictors of mpg would probably help produce a better model.

Final visualization of all 6 models

```
ggplot(Auto, aes(x = displacement, y = mpg)) +  
  geom_point() +  
  geom_line(aes(y = pred_poly, color = "Polynomial"), size = 1) +  
  geom_line(aes(y = pred_smooth, color = "Smoothing Spline"), size = 1) +  
  geom_line(aes(y = pred_cubic, color = "Cubic Spline"), size = 1) +  
  geom_line(aes(y = pred_natural, color = "Natural Spline"), size = 1) +  
  geom_line(aes(y = pred_local, color = "Local Regression"), size = 1) +  
  geom_segment(data = step_data, aes(x = x_start, xend = x_end, y = y, yend = y, color = "Step Function"),  
              linewidth = 1) +  
  scale_color_manual(values = c("Polynomial" = "blue",  
                                "Smoothing Spline" = "orange",  
                                "Cubic Spline" = "green",  
                                "Natural Spline" = "purple",  
                                "Local Regression" = "brown",  
                                "Step Function" = "red")) +  
  labs(title = "All 6 Models", x = "Displacement (cubic inches)", y = "Miles per Gallon", color = "Model")
```



```
rm(list = ls())
```

Ryan Individual Section

```
library(gam)
```

Loading required package: foreach

Loaded gam 1.22-7

```
data(Auto)
set.seed(0)
```

2. Decide who will model which X variable individually and when they need to be done to contribute to the team GAM.

I will do weight

Emery will do acceleration

Jake will do displacement

What individuals need to do

1. Given your team's dataset and your X variable, fit a polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and a local regression (six models).

Polynomial Regression

```
cv_error_poly <- rep(NA, 10)

for (d in 1:10) {
  fit <- glm(mpg ~ poly(weight, d), data = Auto)
  cv_error_poly[d] <- cv.glm(Auto, fit)$delta[1]
}

cv_error_poly

[1] 18.85161 17.52474 17.57811 17.62324 17.62822 17.69418 17.66695 17.76456
[9] 18.35543 18.59401
```

```
poly_reg <- lm(mpg ~ poly(weight, 5), data = Auto)
summary(poly_reg)
```

Call:

```
lm(formula = mpg ~ poly(weight, 5), data = Auto)
```

Residuals:

Min	1Q	Median	3Q	Max
-12.8872	-2.7327	-0.3687	1.8744	16.3456

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	23.4459	0.2114	110.906	< 2e-16 ***
poly(weight, 5)1	-128.4436	4.1856	-30.687	< 2e-16 ***
poly(weight, 5)2	23.1589	4.1856	5.533	5.82e-08 ***
poly(weight, 5)3	0.2204	4.1856	0.053	0.958
poly(weight, 5)4	-2.8079	4.1856	-0.671	0.503
poly(weight, 5)5	3.8301	4.1856	0.915	0.361

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.186 on 386 degrees of freedom
 Multiple R-squared: 0.7161, Adjusted R-squared: 0.7124
 F-statistic: 194.7 on 5 and 386 DF, p-value: < 2.2e-16

Step Function Regression

```
cv_error_step <- rep(NA, 10)

for (k in 2:10) {
  Auto$weight_cut <- cut(Auto$weight, k)
  fit <- glm(mpg ~ weight_cut, data = Auto)
  cv_error_step[k] <- cv.glm(Auto, fit)$delta[1]
}

cv_error_step
```

```
[1]      NA 31.75518 23.74603 22.14387 19.63936 17.84493 18.72716 19.51598
[9] 18.74958 19.02113
```

```
step_reg <- lm(mpg ~ cut(weight, breaks = 6), data = Auto)

summary(step_reg)
```

Call:

```
lm(formula = mpg ~ cut(weight, breaks = 6), data = Auto)
```

Residuals:

Min	1Q	Median	3Q	Max
-14.603	-2.486	-0.423	1.635	16.901

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	32.6033	0.4433	73.55	<2e-
16				
cut(weight, breaks = 6)(2.2e+03,2.79e+03]	-6.1043	0.6082	-10.04	<2e-
16				
cut(weight, breaks = 6)(2.79e+03,3.38e+03]	-11.1170	0.6624	-16.78	<2e-
16				
cut(weight, breaks = 6)(3.38e+03,3.96e+03]	-15.1324	0.6940	-21.80	<2e-
16				

```
cut(weight, breaks = 6)(3.96e+03,4.55e+03] -18.2380      0.7466  -24.43  <2e-
16
cut(weight, breaks = 6)(4.55e+03,5.14e+03] -20.2283      1.1409  -17.73  <2e-
16
```

```
(Intercept) ***
cut(weight, breaks = 6)(2.2e+03,2.79e+03] ***
cut(weight, breaks = 6)(2.79e+03,3.38e+03] ***
cut(weight, breaks = 6)(3.38e+03,3.96e+03] ***
cut(weight, breaks = 6)(3.96e+03,4.55e+03] ***
cut(weight, breaks = 6)(4.55e+03,5.14e+03] ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 4.205 on 386 degrees of freedom

Multiple R-squared: 0.7134, Adjusted R-squared: 0.7097

F-statistic: 192.2 on 5 and 386 DF, p-value: < 2.2e-16

Cubic Spline

```
cv_error_spline <- rep(NA, 10)

for (df in 3:10) {
  fit <- glm(mpg ~ bs(weight, df=df), data = Auto)
  cv_error_spline[df] <- cv.glm(Auto, fit)$delta[1]
}
```

Warning in bs(weight, degree = 3L, knots = numeric(0), Boundary.knots =
c(1649L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = numeric(0), Boundary.knots =
c(1649L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = numeric(0), Boundary.knots =
c(1613L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = numeric(0), Boundary.knots =
c(1613L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = 2800, Boundary.knots = c(1613L, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = 2800, Boundary.knots = c(1613L, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = 2807, Boundary.knots = c(1649L, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = 2807, Boundary.knots = c(1649L, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2395, 3329), Boundary.knots =
c(1613L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases
Warning in bs(weight, degree = 3L, knots = c(2395, 3329), Boundary.knots =
c(1613L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = c(2401, 3336), Boundary.knots =
c(1649L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases
Warning in bs(weight, degree = 3L, knots = c(2401, 3336), Boundary.knots =
c(1649L, : some 'x' values beyond boundary knots may cause ill-conditioned
bases

Warning in bs(weight, degree = 3L, knots = c(2224.5, 2800, 3611),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = c(2224.5, 2800, 3611),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2227, 2807, 3616.5),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = c(2227, 2807, 3616.5),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2155, 2582, 3102, 3820),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2155, 2582, 3102, 3820),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2155, 2585, 3121, 3821),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2155, 2585, 3121, 3821),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2125, 2401, 2807, 3336, 3962: some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2125, 2401, 2807, 3336, 3962: some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2125, 2395, 2800, 3329, 3955: some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2125, 2395, 2800, 3329, 3955: some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2082.14285714286,
2288.85714285714, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2082.14285714286,
2288.85714285714, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2074.71428571429,
2282.85714285714, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2074.71428571429,
2282.85714285714, : some 'x' values beyond boundary knots may cause
ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2045, 2224.5, 2517.5, 2800, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2045, 2224.5, 2517.5, 2800, : some
'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(weight, degree = 3L, knots = c(2048.75, 2227, 2529.25, 2807, :
some 'x' values beyond boundary knots may cause ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = c(2048.75, 2227, 2529.25, 2807, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

```
cv_error_spline
```

```
[1]      NA      NA 17.57811 17.63283 17.59625 17.62741 17.77065 17.70893
[9] 17.45472 17.53000
```

```
cubic_spline_model <- lm(mpg ~ bs(weight, df = 7), data = Auto)
summary(cubic_spline_model)
```

Call:

```
lm(formula = mpg ~ bs(weight, df = 7), data = Auto)
```

Residuals:

Min	1Q	Median	3Q	Max
-12.7754	-2.6635	-0.4582	1.8609	16.1218

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	31.371	2.960	10.599	< 2e-16 ***
bs(weight, df = 7)1	5.637	4.255	1.325	0.186073
bs(weight, df = 7)2	-1.608	2.841	-0.566	0.571641
bs(weight, df = 7)3	-5.899	3.292	-1.792	0.073941 .
bs(weight, df = 7)4	-11.082	3.084	-3.594	0.000368 ***
bs(weight, df = 7)5	-16.548	3.614	-4.579	6.33e-06 ***
bs(weight, df = 7)6	-18.746	3.701	-5.065	6.34e-07 ***
bs(weight, df = 7)7	-19.761	4.012	-4.925	1.26e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.183 on 384 degrees of freedom

Multiple R-squared: 0.718, Adjusted R-squared: 0.7128

F-statistic: 139.7 on 7 and 384 DF, p-value: < 2.2e-16

Natural Spline


```
cv_error_ns <- rep(NA, 10)
```

```
for (df in 3:10) {
  fit <- glm(mpg ~ ns(weight, df=df), data = Auto)
  cv_error_ns[df] <- cv.glm(Auto, fit)$delta[1]
}
```

```
cv_error_ns
```

```
[1]      NA      NA 17.58547 17.67870 17.72421 17.59233 17.42874 17.57322
[9] 17.64938 17.57442
```

```
natural_spline_model <- lm(mpg ~ ns(weight, df = 7), data = Auto)
```

```
summary(natural_spline_model)
```

Call:

```
lm(formula = mpg ~ ns(weight, df = 7), data = Auto)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.3314	-2.5302	-0.4222	1.8281	16.3921

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	31.575	2.038	15.495	< 2e-16 ***
ns(weight, df = 7)1	-6.249	1.933	-3.233	0.001332 **
ns(weight, df = 7)2	-4.599	2.514	-1.829	0.068144 .
ns(weight, df = 7)3	-10.661	2.255	-4.728	3.19e-06 ***
ns(weight, df = 7)4	-13.286	2.378	-5.587	4.39e-08 ***
ns(weight, df = 7)5	-19.001	1.727	-11.005	< 2e-16 ***
ns(weight, df = 7)6	-15.370	4.628	-3.321	0.000982 ***
ns(weight, df = 7)7	-22.327	1.958	-11.401	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.143 on 384 degrees of freedom

Multiple R-squared: 0.7233, Adjusted R-squared: 0.7183

F-statistic: 143.4 on 7 and 384 DF, p-value: < 2.2e-16

Smoothing Spline

```
smooth_fit <- smooth.spline(Auto$weight, Auto$mpg, cv = TRUE)
```

Warning in smooth.spline(Auto\$weight, Auto\$mpg, cv = TRUE): cross-validation with non-unique 'x' values seems doubtful

```
smooth_fit
```

Call:

```
smooth.spline(x = Auto$weight, y = Auto$mpg, cv = TRUE)
```

Smoothing Parameter spar= 0.8477733 lambda= 0.0006595457 (16 iterations)
Equivalent Degrees of Freedom (Df): 10.35552
Penalized Criterion (RSS): 5818.045
PRESS(1.o.o. CV): 17.55403

Local Regression

```
cv_error_loess <- rep(NA, 10)
spans <- seq(0.1, 1, by=0.1)

for (i in 1:length(spans)) {
  fit <- loess(mpg ~ weight, data=Auto, span=spans[i])
  pred <- predict(fit, Auto$weight)
  cv_error_loess[i] <- mean((Auto$mpg - pred)^2)
}
```

```
cv_error_loess
```

```
[1] 15.91016 16.42588 16.62438 16.84477 17.02355 17.21793 17.28223 17.30830
[9] 17.31000 17.30680
```

```
best_span <- spans[which.min(cv_error_loess)]
best_span
```

```
[1] 0.1
```

```
loess_reg <- loess(mpg ~ weight, data = Auto, span = 0.1)

summary(loess_reg)
```

Call:

```
loess(formula = mpg ~ weight, data = Auto, span = 0.1)
```

Number of Observations: 392

Equivalent Number of Parameters: 30.34

Residual Standard Error: 4.19

Trace of smoother matrix: 33.55 (exact)

Control settings:

```
span      : 0.1
degree    : 2
family    : gaussian
surface   : interpolate    cell = 0.2
normalize: TRUE
parametric: FALSE
drop.square: FALSE
```

1. Use CV to determine the optimal tuning parameters for at least three of your models. Sometimes just choosing the tuning parameter (e.g., picking the number of cuts in X for a step function regression) based on your knowledge of Q1 and Q3 makes more sense than using CV?

Done above while fitting the models[^]

1. Compare the CV MSE or other appropriate model summary (e.g., misclassification rate) for your models. Choose the best model method.

```
poly_model <- glm(mpg ~ poly(weight, 5), data = Auto)
cv_poly <- cv.glm(Auto, poly_model)$delta[1]

Auto$weight_bins <- cut(Auto$weight, 6)
step_model <- glm(mpg ~ weight_bins, data = Auto)
cv_step <- cv.glm(Auto, step_model)$delta[1]

cubic_model <- glm(mpg ~ bs(weight, df = 7), data = Auto)
cv_cubic <- cv.glm(Auto, cubic_model)$delta[1]
```

Warning in bs(weight, degree = 3L, knots = c(2155, 2582, 3102, 3820),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause

```
ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = c(2155, 2582, 3102, 3820),
Boundary.knots = c(1613L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases
```

```
Warning in bs(weight, degree = 3L, knots = c(2155, 2585, 3121, 3821),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases
Warning in bs(weight, degree = 3L, knots = c(2155, 2585, 3121, 3821),
Boundary.knots = c(1649L, : some 'x' values beyond boundary knots may cause
ill-conditioned bases
```

```
natural_model <- glm(mpg ~ ns(weight, df = 7), data = Auto)
cv_natural <- cv.glm(Auto, natural_model)$delta[1]
```

```
loess_model <- loess(mpg ~ weight, data = Auto, span = 0.1)
pred_loess <- predict(loess_model, Auto$weight)
cv_loess <- mean((Auto$mpg - pred_loess)^2)
```

```
results <- data.frame(
  Model = c("Poly Regression",
            "Step Function",
            "Cubic Spline",
            "Natural Spline",
            "LOESS"),
  CV_MSE = c(cv_poly,
             cv_step,
             cv_cubic,
             cv_natural,
             cv_loess)
)
```

```
results
```

	Model	CV_MSE
1	Poly Regression	17.62822
2	Step Function	17.84493
3	Cubic Spline	17.77065
4	Natural Spline	17.42874
5	LOESS	15.91016

- At an important or significant value of X (explain why that value of x0 is important or significant), use your model to predict Y at that value.

```
best_model <- loess(mpg ~ weight, data = Auto, span = 0.1)
mean(Auto$weight)
```

```
[1] 2977.584
```

```
predict(best_model, newdata=data.frame(weight=2977.584))
```

```
1
23.05275
```

```
mean(Auto$mpg)
```

```
[1] 23.44592
```

3. Interpret your prediction and generally what the model is telling you about Y.

I selected the mean vehicle weight (2977.6 lbs) as a meaningful value because it represents a typical vehicle in the dataset. Using the best model chosen by lowest MSE across the 6 different options, 15.91, the LOESS model with $\text{span} = 0.1$, the predicted fuel efficiency at this weight is approximately 23.05 miles per gallon. This prediction is very close to the average mpg of the data set which is 23.45 mpg, which suggests that the model produces realistic predictions.

1. Comment on any ethical implications of this work.

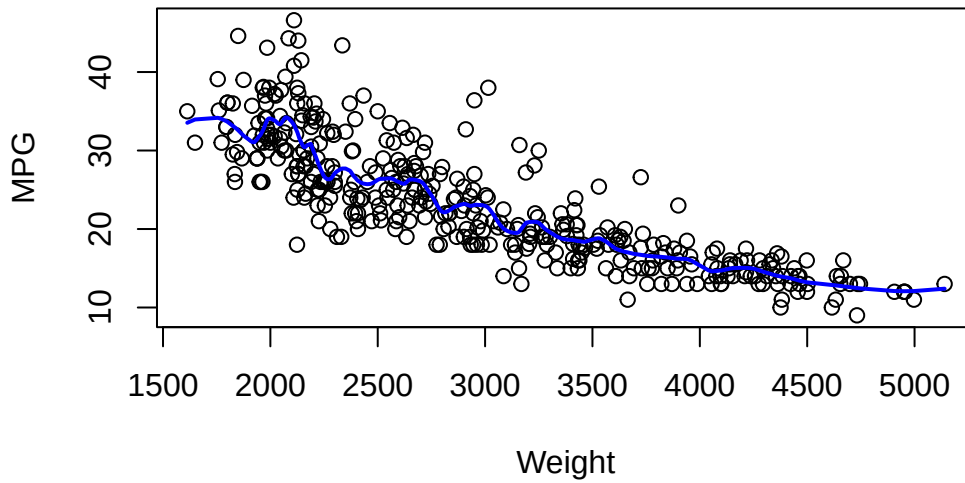
Fuel efficiency is directly related to environmental impact, greenhouse gas emissions, and climate change. Heavier vehicles tend to produce lower fuel efficiency, which contributes to higher carbon emissions. If policymakers and manufacturers use similar modeling techniques to inform regulations and design standards without considering broader social issues, they could disproportionately affect certain consumers and environmental issues. Additionally, this model uses weight as a single predictor, which oversimplifies fuel efficiency. Overreliance on simplified models could lead to misleading conclusions if other important variables are ignored.

1. Optional: Create visualizations of your models or just of the best model.

```
plot(Auto$weight, Auto$mpg,
     xlab = "Weight",
     ylab = "MPG",
     main = "LOESS Fit: MPG vs Weight")

lines(sort(Auto$weight),
      predict(best_model,
              newdata = data.frame(weight = sort(Auto$weight))),
      col = "blue",
      lwd = 2)
```

LOESS Fit: MPG vs Weight



Team Section

1/2. Creating our GAM and Visualizing it

```
library(gam)
set.seed(0)
dat <- na.omit(Auto)
gam_fit <- gam(
  mpg ~ lo(weight, span = 0.1) +
    s(displacement, df = 10) +
    s(acceleration),
  data = dat
)
```

Warning in general.wam(x, z, wz, fit\$smooth, which, fit\$smooth.frame, bf.maxit,
: general.wam convergence not obtained in 30 iterations

```
summary(gam_fit)
```

```
Call: gam(formula = mpg ~ lo(weight, span = 0.1) + s(displacement,
  df = 10) + s(acceleration), data = dat)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-10.6716	-2.2086	-0.2727	1.7278	17.3079

(Dispersion Parameter for gaussian family taken to be 15.7984)

Null Deviance: 23818.99 on 391 degrees of freedom

Residual Deviance: 5637.777 on 356.8576 degrees of freedom

AIC: 2229.798

Number of Local Scoring Iterations: 1

Anova for Parametric Effects

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
lo(weight, span = 0.1)	1.00	16779.1	16779.1	1062.0737	< 2.2e-16	***
s(displacement, df = 10)	1.00	302.7	302.7	19.1598	1.581e-05	***
s(acceleration)	1.00	116.6	116.6	7.3831	0.006905	**
Residuals	356.86	5637.8	15.8			

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

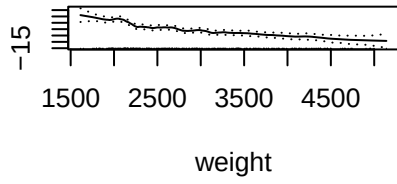
Anova for Nonparametric Effects

	Npar	Df	Npar F	Pr(F)	
(Intercept)					
lo(weight, span = 0.1)	19.1	3.0443	2.31e-05	***	
s(displacement, df = 10)	9.0	3.1958	0.000979	***	
s(acceleration)	3.0	2.0518	0.106327		

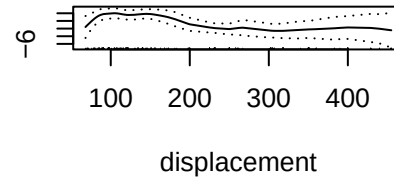
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
par(mfrow = c(2,2))
plot(gam_fit, se = TRUE)
par(mfrow = c(1,1))
```

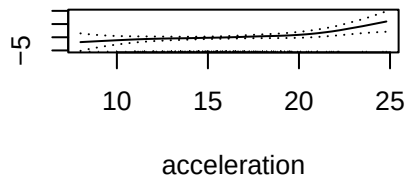
lo(weight, span = 0.1)



s(displacement, df = 10)

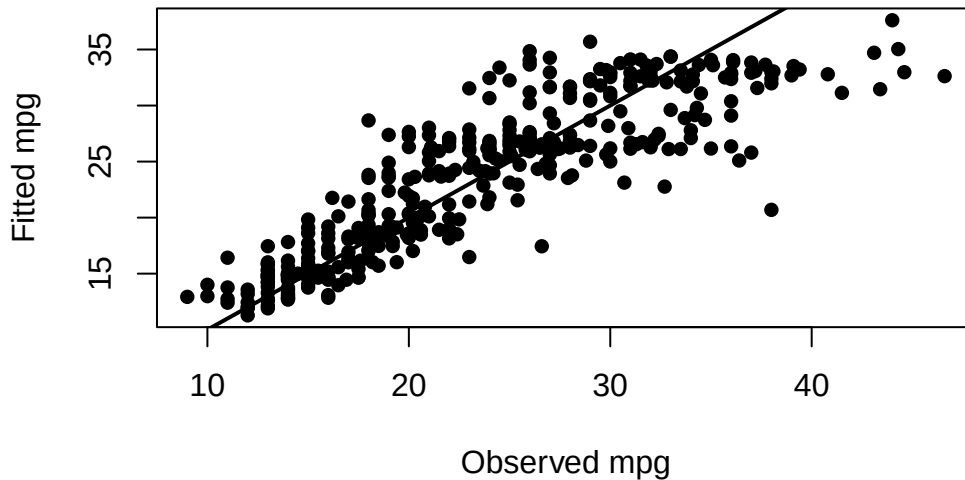


s(acceleration)



- Observed vs fitted plot

```
plot(dat$mpg, fitted(gam_fit),  
     xlab = "Observed mpg",  
     ylab = "Fitted mpg",  
     pch = 16)  
abline(0,1,lwd=2)
```

Part 3. Predicted Value

```
new_data <- data.frame(  
  weight = mean(dat$weight),  
  displacement = mean(dat$displacement),  
  acceleration = mean(dat$acceleration)  
)  
  
gam_prediction <- predict(gam_fit, newdata = new_data)  
  
cat("Predicted mpg for an average car:", round(gam_prediction, 2))
```

Predicted mpg for an average car: 22.09

We evaluated the fitted GAM at the mean values of weight, displacement, and acceleration. This is a significant value since it represents a typical car in the dataset. Using these average characteristics, the model predicts a fuel efficiency of 22.09 miles per gallon. Because the GAM allows for flexible nonlinear relationships between mpg and each predictor, this estimate reflects curvature in how weight, displacement, and acceleration influence fuel efficiency rather than assuming purely linear effects.

Part 4. Comparison to Linear Method

We compared the GAM to a lasso linear regression model using MSE under LOOCV.

```
library(glmnet)
```

Loading required package: Matrix

Loaded glmnet 4.1-10

```
library(gam)
x <- model.matrix(mpg ~ weight + displacement + acceleration, data = dat)[, -1]
y <- dat$mpg

# LOOCV lasso
cv_lasso <- cv.glmnet(x, y, alpha = 1, nfolds = nrow(dat))
```

Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per fold

```
best_lambda <- cv_lasso$lambda.min
mse_lasso_loocv <- cv_lasso$cvm[cv_lasso$lambda == best_lambda]

# LOOCV GAM
n <- nrow(dat)
loocv_errors_gam <- rep(NA, n)

for (i in 1:n) {
  gam_temp <- suppressWarnings(gam(
    mpg ~ lo(weight, span = 0.1) +
      s(displacement, df = 10) +
      s(acceleration),
    data = dat[-i, ]
  ))
  pred <- predict(gam_temp, newdata = dat[i, ])
  loocv_errors_gam[i] <- (dat$mpg[i] - pred)^2
}
```

Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : eval 5140

Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : upperlimit 5013.9

```
Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : extrapolation not allowed with blending
```

```
Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : eval 1613
```

```
Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : lowerlimit 1631.5
```

```
Warning in gam.lo(data[["lo(weight, span = 0.1)"]], z, w, span = 0.1, degree = 1, : extrapolation not allowed with blending
```

```
mse_gam_loocv <- mean(loocv_errors_gam)
cat("LOOCV MSE for Lasso:", round(mse_lasso_loocv, 3), "\n")
```

LOOCV MSE for Lasso: 18.589

```
cat("LOOCV MSE for GAM:", round(mse_gam_loocv, 3), "\n")
```

LOOCV MSE for GAM: 17.285

The GAM MSE (17.285) was lower than lasso MSE (18.589) which was expected since the GAM allowed for non-linear relationships. During single variable analysis, we observed some non-linear relationships between mpg and predictor, such as displacement. Therefore, our GAM likely has lower bias than the lasso linear fit.

Team section contributions

- Emery compiled the individual sections, fit the GAM, and plotted observed vs fitted values.
- Ryan evaluated the GAM at the mean values of each predictor.
- Jake compared the GAM to a lasso linear regression model using LOOCV MSE.